

GwenLaya v4: a pre-registered study of answering only when a check can back the answer

Interim report after the first night run: registered design, deviations,
descriptive results only, and no completed pre-registered test

Xavier Callens
AutoevolveAI / ANSE open-source project

8 October 2026 (revision v3.3.1, branch `fix/math-and-rescore`; supersedes v3.3.0)

Abstract

GwenLaya is a system design in which a small open model (Qwen3.5 family) answers a programming or mathematics question only when an executed check or a calibrated confidence score supports the answer, and otherwise abstains or escalates to a larger tier. We pre-registered fifteen hypotheses (two primary: fewer confident-wrong answers than log-probability selective prediction at matched coverage, H1; lower cost per correct answered item than always using the largest tier with the gate, H3) before any GwenLaya run. This report covers the first, budget-limited night run. **No pre-registered hypothesis test was completed.** The scored night data are 80 items from one arm that is not a registered arm (Qwen3.5-2B, q4_K_M, CPU, no gate, no calibrator). **Revision v3.3.1 corrects v3.3.0** (Section 2): the v3.3.0 Rust result (0/26) was an artefact of our sandbox, in which no Rust program could link (D27); after the fix, and a check of the harness on hand-written correct and wrong solutions, the same 26 Rust generations pass the hidden checks in 6 cases. Python is unchanged: 11/28 passed the hidden checks (pass rate 0.393, 95% cluster bootstrap CI 0.214–0.571) while the raw exp(mean log-prob) confidence averaged 0.896, so it was badly calibrated (ECE 0.519, CI 0.392–0.689); its AUROC was 0.717 with a CI (0.483–0.917) that includes 0.5. The 26 night math generations remain invalid (they went through the code prompt path, D28) and none passes under the registered boxed-only rule. These are descriptive numbers on public benchmarks that the models have probably seen in pretraining, not tests. We also report a descriptive, unregistered TPU run (one v5e chip, greedy, one sample) of Qwen3.5-2B, Qwen3.5-4B and Qwen2.5-Coder-1.5B on the same 80 tasks, re-scored on our machine because scoring on the TPU host was invalid for Python; Qwen3.5-2B in bf16 on TPU gives the same Python and Rust pass counts as the q4 CPU arm (not the same items in every case), a consistency check rather than a test. The budget hypothesis (H13) is not refuted on an interim basis (recorded spend \$1.54, estimated from catalog list prices), but the spend ledger enforces it by construction, so this is not evidence about the system. Review also found that the HumanEval+ gate payloads of the night task files contained the hidden tests; no gate ran, so no number is affected, and the files have been rebuilt (D25). No TPU LoRA path was found for dense Qwen3.5 or Qwen3.8. We report the GCP spend actually recorded and every deviation from the registration, and begin by restating the retraction of a v3 claim and the failed v3 gate.

1 Retraction and the v3 gate failure, stated first

Retraction. Revision 3.1.0 of the predecessor report [2] claimed that the low-tier optimizer improved pass rates by at least +8.0 percentage points. That claim rested on a hand-written results file with invented numbers (A0 33%, A3 100%). The file was fabricated, it has been deleted, and it is not evidence (`results/PREREGISTRATION.md`). The claim is retracted. Nothing in this paper depends on it.

The measured v3 gate failed. The pre-registered v3 gate was $A3 - A0 \geq +8.0$ points pass@1 on MBPP-sanitized with `qwen2.5-coder:1.5b`. Both measured runs **FAIL** that gate:

- $n = 60$, CPU: $A3 - A0 = +6.67$ points, 95% CI [1.67, 13.33], 4 gained / 0 lost, exact McNemar $p = 0.125$; verdict FAIL (`results/results_n60_cpu.json`).
- $n = 257$, L4 GPU: $A0$ 64.2 %, $A3$ 71.98 %, $A3 - A0 = +7.78$ points, 95% CI [4.67, 11.28], 20 gained / 0 lost; verdict FAIL against +8.0. This file’s `git_commit` field is empty (`results/results_n257_l4_gpu.json`).

At $n = 60$ the difference is not significant by the exact McNemar test ($p = 0.125$, 4/0 discordant pairs); the percentile-bootstrap interval there is not informative, because when every discordant pair favours $A3$ a resample can reach $\Delta \leq 0$ only by drawing no discordant item at all, so its lower bound is positive almost by construction. Only the $n = 257$ run (20/0 discordant; the file rounds the exact McNemar p to 0.0, i.e. $p < 0.001$) supports a nonzero effect. In both runs the registered +8.0 threshold was missed, and we report both as FAIL. The same file shows the v3 gate’s weakness that motivates v4: of answers the gate called VERIFIED (coverage 80.93 %), only 88.94 % passed the hidden tests. By arithmetic on those two file values (not a measurement), about 8.95 % of all problems were confident-wrong under v3.

2 Corrections in v3.3.1

Revision v3.3.0 of this report contained a wrong Rust result caused by our own harness. This revision corrects it and records four new deviations (D27–D30, Table 1). All were found and fixed after the night generations existed; no registered test has run, so none was written with a test result in view.

- **D27, Rust harness (v3.3.0 Rust result withdrawn).** Inside the bubblewrap sandbox, `rustc`’s linker `cc` resolves through `/etc/alternatives`, which the sandbox did not mount, so *every* Rust program failed to link. The v3.3.0 Rust result (0/26 passing the hidden checks) was therefore a harness artefact, not a model result. While fixing it we also found that the harness wrote its pass nonce into `main.rs`, so a candidate could read its own source or binary and forge a pass; we reproduced this and closed it (nonce passed on stdin; separate compile and run sandboxes; no `/proc` in the run sandbox; `unsafe`, `link_section` and `asm` rejected). `assert_ne!` lines are now counted alongside `assert_eq!`. Validation (`results/gwenlaya_v4/rust_harness_validation.json`): hand-written reference solutions for 25 of the 26 night Rust tasks all pass (25/25), and 9 deliberately wrong solutions all fail (9/9). Re-scoring the unchanged night generations gives Rust 6/26 passing the hidden checks (Section 5). The compile failures that were examined are model errors (for example indexing a `str` by `usize`, or calling a non-existent `.unique()` method).
- **D28, math prompt path and scoring.** The night math generations went through the code path (Python system prompt, “‘python prefill, “‘ stop), so they remain invalid; this conclusion is unchanged. The code is fixed for future runs: a prose math prompt ending in `\boxed{}`, non-thinking mode for Qwen3.x through an empty think block, and the registered boxed-only rule as the scoring default (the last-number fallback of D24 is now opt-in only). Under boxed-only, all 26 night math generations are UNDECIDED (none contains `\boxed`). A self-consistency key bug (the math vote key used the extraction method instead of the answer) was also fixed; self-consistency (B4) did not run (D9), so no reported number is affected.
- **D29, Python harness environment.** With a `uv`-managed interpreter the `venv`’s `python` symlink chain passed through a directory not bound into the sandbox, so Python could not start inside bubblewrap on the re-scoring machine. Fixed by recreating the interpreter symlinks. The Python result is unchanged (11/28).

- **D30, TPU scoring and an unregistered TPU run.** A third TPU run generated all 80 night tasks for three models (Section 7). Python scoring launched from inside the vLLM process on the TPU host failed for every item, so all TPU generations were re-scored on our machine with the same checkers. This run is descriptive and was not a registered arm.

3 What GwenLaya is and what was registered

The full registration is `docs/GWENLAYA_PREREGISTRATION.md` (revision 2, design freeze; sha256 of each revision in `docs/PREREG_FREEZE.txt`). In brief:

- **System (GL).** Generate with a small Qwen3.5 tier; run a *gate-visible* executed check that, by design, never sees the scoring tests (in the night task files this was violated for HumanEval+, whose gate payloads contained the whole hidden suite; no gate ran, and the files were rebuilt, D25, Section 5.3); score the candidate with a calibrated confidence head (a LoRA head on the `convaiinnovations/laya` encoder); answer VERIFIED, LIKELY_CORRECT(p), abstain, or escalate to a larger tier. The label for calibration is the outcome on the *hidden* checks, so VERIFIED is never treated as ground truth.
- **Eval pool E.** HumanEval+ and MBPP+ [11] (Python, 542 items), MultiPL-E Rust [3] (510), MATH-500 [7, 10] (500), and miniF2F Lean 4 (244, outside the pooled analysis).
- **Arms.** B1 smallest tier alone; B2 largest tier alone; B3 log-probability selective prediction [4]; B4 self-consistency; B5 largest tier with the v3-style gate only; B6 Laya zero-shot; LR and a shuffled-label negative control SH.
- **Primary family** (Holm [8], $m = 2$; a primary that does not run enters with $p = 1$, so the other is tested at 0.025): H1, CWR(GL)–CWR(B3) at matched coverage, exact one-sided McNemar [12] plus a cluster-stratified paired bootstrap, MEI 3.0 points; H3, cost per correct answered item, GL vs B5, MEI 25 % reduction, with answered accuracy non-inferior by 2.0 points.
- **Secondary:** calibration (H4 ECE [6], H5 Laya vs LR AUROC, H6 split-conformal selective error [1]); quantization (H8); training (H9); systems (H13 budget, H14 endpoint latency, H15 preemption). **Exploratory:** H2, H7, H10–H12.

Hypothesis wording and MEIs are unchanged. The H3 cost unit (D3), the H5 comparator (D5, if the frozen-embedding probe replaces the LoRA head) and the size of the primary Holm family (D22, made after the L2 generations existed and reverted to the registered $m = 2$ in D26) are deviations. Throughout, VERIFIED keeps its registered meaning (all gate-visible checks passed); the outcome on the hidden checks is written PASS_HIDDEN / FAIL_HIDDEN / UNDECIDED.

4 The night run and its deviations

The night of 2026-10-07/08 was meant to run the primaries cheaply. Two constraints shaped it: the total GCP budget (\$50 hard, \$45 working ceiling for this programme, \$40 planned), and the fact that the project’s only GPU quota unit (GPUS_ALL_REGIONS limit 1) was held all night by a VM that is not part of this project and that we did not touch. All generation therefore ran on a shared 8-core CPU, and the scope was cut. Every deviation is in `docs/DEVIATIONS.md` and is summarised in Table 1. D1–D20 were written before generation; D21 during L2; D22–D23 after the L2 generations existed and before any registered test; D24–D26 after the external review of the first draft of this report; D27–D30 after release v3.3.0 (Section 2). No registered test has run, so none of these was written with a test result in view.

Table 1: Deviations from the registration for the night run (from docs/DEVIATIONS.md). Reason codes: B = budget, G = no GPU slot, T = CPU wall clock, I = implementation or defect.

#	Registered	Night run	Why
D1	E = 1,552 items on L4	E-night: seeded prefix, 80 items (python 28, rust 26, math 26)	G, T
D2	tiers 2B/4B/9B/27B	2B and 4B only (B2/B3/B5 on 4B)	G, T
D3	H3 cost in L4 GPU-seconds	CPU-seconds on this machine	G
D4	separate calibration set C	5-fold cross-fitting on E-night by source cluster	T; C was not label-matched
D5	Laya LoRA r=16 heads	LoRA only if CPU timing fits, else frozen-embedding probe	T
D6	llama.cpp server	llama.cpp b11476 CPU; Ollama not used (no logprobs)	I
D7	Lean 4 (E, C, H11)	dropped tonight	T, B
D8	SFT, DPO, S4b-S5	deferred; H9 deferred, H10-H12 dropped	G, B
D9	B4 self-consistency, second seed	dropped	B, T
D10	GSM8K, AIME25, LiveCodeBench, MBPP anchor, 0.8B	dropped	B, T
D11	H8 on full E	300-item 9B subsample only if GPU; no TOST	G
D12	endpoint, 4-6 cold samples	4 samples, only after G1	G, B
D14-D15	-t 8; 256-token tok/s probe	-t 4; 64 tokens	T (foreign CPU load)
D16	-	fix: multi-letter words in math checker return undecided	I
D17	reference passes its own checks	Rust: typecheck only (no reference solution)	I
D18	decontaminate T, C	all 170 MBPP-sanitized calib items removed; no T/C	I
D19	n from measured throughput	n from measured tok/s and an <i>assumed</i> 196.6 tokens/item	T
D20	prompt text not fixed	gate-visible tests and prompts frozen in the task file	I
D21	-	logprobs and per-call CPU-seconds captured	I
D22	random.Random(0) bootstrap; primary Holm $m = 2$	numpy default_rng(0), 10,000 resamples; Holm over the primaries that ran ($m = 1$ or 2), a post-data decision-rule change, reverted in D26	I
D23	-	scored the 80 L2 generations; single-arm descriptives added	I
D24	math: boxed answer only; un-parseable = UNVERIFIED	scorer fell back to ####, then to the last number in the text; all 26 math items were scored that way; registered boxed-only rescoring now reported too	I (found in review)
D25	gate never sees the scoring checks	HumanEval+ gate payloads held the full hidden suite; rebuilt with an AST rewrite, 5 primary items excluded; no gate ran	I (found in review)
D26	Holm $m = 2$	restored ($p = 1$ for a primary that did not run); per-domain descriptives, Python headline	I (post-L2)
D27	Rust: compile and run all hidden asserts	sandbox could not link any Rust program (v3.3.0 Rust 0/26 was an artefact); fixed, nonce forgery closed, assert_ne! counted; validated 25/25 correct, 9/9 wrong; night Rust re-scored	I (post-v3.3.0)
D28	math: boxed answer, math prompt	night math went through the code prompt path (still invalid); prose \boxed{} prompt, Qwen3.x non-thinking, boxed-only scoring by default; self-consistency key bug fixed	I (post-v3.3.0)
D29	Python checks in bwrap	uv-managed interpreter could not start in the sandbox; symlinks recreated; Python unchanged	I (post-v3.3.0)
D30	-	unregistered descriptive TPU run (3 models, 80 tasks); TPU-host Python scoring invalid, all TPU rows re-scored locally	I, G (post-v3.3.0)

Benchmarks reduced. To stay within budget and wall clock we ran no benchmark beyond the three primary domains, on 80 of their 1,541 included items, with two tiers instead of four. Lean 4, GSM8K, AIME25, LiveCodeBench, the MBPP continuity anchor, the 0.8B tier, self-consistency and the second seed were dropped (D7–D10). The registered design for all of them stays in the registration and is not reported here as if it had run.

Power was known to be insufficient before generation. The freeze addendum computed the power of H1’s exact McNemar test at $n = 80$ before any item was generated: 0.052 at the 3-point MEI and 0.261 at a 6-point effect ($\alpha = 0.025$). A non-significant H1 at this n would therefore have been inconclusive, not evidence of no effect.

What actually ran. L0 (environment, throughput), L1 (manifest, freeze addendum, decontamination), L2 (2B generation on all 80 E-night items, 72 minutes wall), and scoring of those 80 generations. L3 (4B generation) was held because the L2 review found a blocking defect (Section 5.3). G1 (GPU) and E1 (endpoint) never ran: the GPU slot never freed. No Laya head, calibrator, threshold or second arm exists. After v3.3.0 the same 80 generations were re-scored with the fixed harnesses (D27–D29; no new CPU generation), and an unregistered TPU run generated the 80 tasks for three models (D30, Section 7).

5 Results

5.1 Per-hypothesis verdicts

Table 2 reproduces `docs/RESULTS_VERDICT.md`. For every hypothesis that did not run, the probability column is the **red-team prior** written in the review before the run (`docs/PROPOSAL_REVIEW.md`, copied into `docs/DEVIATIONS.md`). It is *not* an estimate from data. This includes H13, the only hypothesis whose decision rule could be evaluated: its row shows the prior, because the check holds largely by construction and we give no estimated probability for it.

Table 2: Verdicts after the night run. “Prior” = red-team prior that the hypothesis would be supported, written before any data; not updated by evidence.

H	Role	Verdict	$P(\text{true})$	Basis
H1	primary	not run	0.45 (prior)	no GL or B3 arm; power at MEI with $n = 80$ was 0.05
H3	primary	not run	0.50 (prior)	no GL or B5 arm; cost unit would be CPU-s (D3)
H4	secondary	not run	0.15 (prior)	no cross-fitted scores; no Laya head
H5	secondary	not run	0.25 (prior)	same
H6	secondary	not run	0.40 (prior)	same
H8	secondary	not run	0.30 (prior)	needs GPU (D11)
H9	secondary	deferred	0.25 (prior)	D8
H13	systems	not refuted (interim)	0.75 (prior)	ledger \$1.54 (catalog estimate); holds by construction, Sec. 8
H14	systems	not run	0.50 (prior)	needs E1, which needs the GPU (D12)
H15	systems	not run	0.70 (prior)	offline test is not the registered SIGKILL check
H2	exploratory	not run	0.08 (prior)	no GL or B5 arm
H7	exploratory	not run	0.10 (prior)	gate field is null in every scored row
H10	exploratory	dropped	0.05 (prior)	D8
H11	exploratory	dropped	0.12 (prior)	D7, D8
H12	exploratory	dropped	0.12 (prior)	D8

H13 (budget). Decision rule: ledger total $\leq \$40$ planned and $\leq \$50$ hard, and no stage over $1.25\times$ its cap twice. The machine verdict (`numbers_v4.json`) is “not refuted”: the ledger has 5 lines, all with a `usd_estimate`, with a USD total of 1.5399. The two TPU smoke-test lines were first recorded without `usd_estimate` and `resource`; they were priced on 2026-10-08 from the Cloud Billing Catalog list prices (on-demand v5e \$1.20 per chip-hour, us-central1; spot v5e \$0.494237 per chip-hour, us-west4). The two later TPU runs were priced at the on-demand list price as recorded (Section 8). These are list-price estimates; the billed amount is TBD until the billing export is read. Two caveats limit what this means. First, the ledger refuses any stage that would exceed its cap, so H13 holds largely by construction; the red-team review called its support “close to tautological” (prior 0.75), and it is not evidence about the GwenLaya system. Second, H13 is a whole-study claim, the planned GCS ledger `runs/LEDGER.jsonl` does not exist, and later nights will add spend, so the verdict is interim. An earlier draft of this report gave $P(\text{true}) = 0.97$; that was author judgement with no statistical basis, and it has been removed.

H15 (preemption). `tests/test_run_study.py::test_resume_after_kill_mid_run` passes, but it simulates a death by appending a torn line; it sends no SIGKILL and measures no lost wall time. It is weak support that resume is item-granular, not the registered verification, so the probability stays at the prior.

5.2 Descriptive single-arm result (not a pre-registered test)

The scored arm `base` is Qwen3.5-2B q4_K_M on llama.cpp [5], greedy, thinking off, `max_new_tokens` 1024, with no gate, no calibrator and no escalation. It is not one of the registered arms. Confidence is the raw $\exp(\text{mean token log-prob})$, uncalibrated and not cross-fitted. Intervals are 95% cluster-stratified bootstrap percentile intervals (10,000 resamples, seed 0). Every number below is read from `papers/numbers_v4.json` (`single_arm.by_domain.*`, `single_arm.boxed_only.*`, `arm.base.*`), written by `scripts/analyze_study.py`; its tables are reproduced unedited in Appendix A. In v3.3.1 these files were regenerated from the re-scored rows `results/gwenlaya_v4/night_L2fix/rows.jsonl` (same generations as v3.3.0, fixed Rust harness, boxed-only math scoring; D27–D29).

Outcome labels. The per-row outcome reported here is the result of the *hidden* checks: PASS_HIDDEN, FAIL_HIDDEN or UNDECIDED (`rows.jsonl` field `score` = VERIFIED, FAILED or UNVERIFIED). It is *not* the registered VERIFIED, which is the gate’s verdict on the gate-visible checks. The gate verdict was not recorded in any of the 80 rows (`gate` is null in all of them), so this run contains no VERIFIED in the registered sense.

Why Python is the headline. Pooling the three domains would mix usable domains with one that is not (Section 5.3): the 26 math generations are invalid, so a pooled AUROC or ECE would partly describe that defect, not the confidence signal. Python ($n = 28$) was chosen as the headline in D26, before the Rust harness was fixed, and we keep that choice rather than change the headline after seeing the re-scored Rust numbers. Rust ($n = 26$) is now a second descriptive domain, with the caveat that its harness was validated only after the generations were scored (D27). Math and the pooled numbers appear only as labelled rows of Table 3.

Python. 11 of 28 items passed the hidden checks and 17 failed (0 undecided), a pass rate of 0.393 (CI 0.214–0.571). The mean raw confidence was 0.896 (CI 0.875–0.917), so the score is badly calibrated (ECE 0.519, CI 0.392–0.689; Brier 0.476, CI 0.334–0.617). The AUROC point estimate was 0.717 (AURC 0.438, CI 0.221–0.712), but its interval (0.483–0.917) includes 0.5, so with $n = 28$ we cannot say that it ranks correct answers above wrong ones. The confident-wrong rate, counting every answered item that did not pass, was 0.607 (CI 0.429–0.786). Cost per passing item was 330 server CPU-seconds (CI 174–726).

Table 3: Raw confidence of arm **base**, descriptive (from `papers/numbers_v4.json`). Pass = PASS_HIDDEN. 95% cluster bootstrap CI in brackets. Python is the headline; Rust is descriptive with a harness validated after scoring (D27); the math and pooled rows are sensitivity analyses and are not valid estimates. Cost = server CPU-seconds per passing item (D3, D21). TBD = undefined (no passing item). v3.3.1 values (re-scored rows).

Subset	n	Pass rate	Mean conf.	ECE	AUROC	Cost / pass
Python (headline)	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
Rust (descriptive, D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
<i>Sensitivity only, not valid estimates</i>						
Math (invalid), boxed-only ^a	26	0.000	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD	TBD
Pooled (boxed-only math)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

^a Registered rule, now the scoring default (D28): no `\boxed` = UNVERIFIED (UNDECIDED here). The unregistered last-number fallback of v3.3.0 (D24) is no longer applied, so the machine table’s “as scored” and “boxed-only” math rows, and its two pooled rows, now coincide (Appendix A).

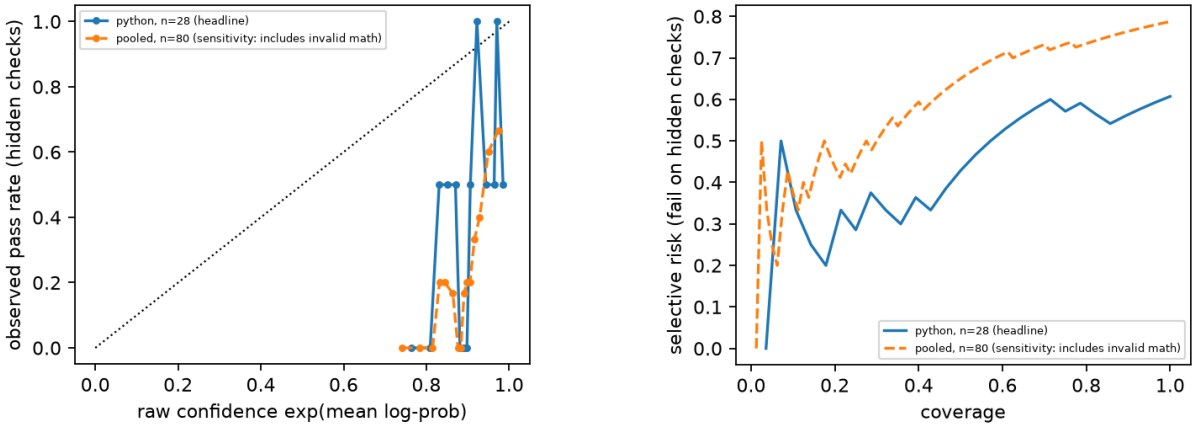


Figure 1: Arm **base**, raw confidence (v3.3.1 re-scored rows). Solid: Python, $n = 28$ (headline). Dashed: all 80 items, a sensitivity curve only, because it contains the invalid math items (Section 5.3). Left: reliability diagram (equal-mass bins). Right: risk-coverage curve; risk = share failing the hidden checks. Descriptive only; not cross-fitted.

Rust (descriptive; corrected in v3.3.1). With the fixed harness (D27), 6 of 26 items passed the hidden checks and 20 failed (0 undecided), a pass rate of 0.231 (CI 0.077–0.385). This replaces the v3.3.0 figure of 0/26, which was a linking failure of our sandbox. Mean raw confidence was 0.894 (CI 0.882–0.906), so here too the score is badly calibrated (ECE 0.675, CI 0.549–0.813; Brier 0.605, CI 0.477–0.725). The AUROC point estimate was 0.875 (CI 0.709–0.990; AUC 0.557, CI 0.321–0.851); with 6 passing items, one seed, an uncalibrated score and probable benchmark exposure we do not read this as evidence that the score ranks Rust answers well. Cost per passing item was 494 server CPU-seconds (CI 261–1552). The harness was validated after these generations existed, on hand-written solutions (25/25 correct pass, 9/9 wrong fail), not on an independent reference set.

Math (not interpretable). Under the registered boxed-only rule, now the default (D28), all 26 math items are UNDECIDED (0 PASS_HIDDEN, 0 FAIL_HIDDEN): none of the generations contains `\boxed`, because they were produced through the code prompt path. The last-number fallback that v3.3.0 reported as “as scored” (D24) is no longer applied. Pooled over 80 items the pass rate is 0.212 (CI 0.125–0.300); it mixes the invalid math subset with the two code domains and is not interpreted.

Contamination. HumanEval+, MBPP+, MultiPL-E and MATH-500 are public and were very probably seen in Qwen3.5 pretraining (registration Section 10, “Residual risk, stated”). The absolute pass rates above are therefore not clean capability estimates; the registration claims only paired, within-model deltas, and none was measured. Memorised items would also tend to have high log-probability and pass, which can inflate AUROC differently by score type (review finding F12).

What we read from this, and no more: on 28 Python and 26 Rust items the raw log-probability of a 2B model at q4 sits near 0.9 while well under half of the answers pass, so it is not usable as a probability without calibration. This is consistent with the motivation for a calibrated head, but it does not test H1, H4 or H5.

5.3 Defects found, and why some numbers are invalid

1. **Math prompt path (blocking).** All 26 math completions were produced with the Python code system prompt and a “python prefill, because the prompt builder falls back to the python tag for any domain it does not list. 0 of 26 contain \boxed. The frozen user prompt asked for a boxed answer, but the system prompt and prefill overrode it. The math pass rate says nothing about the model’s math ability. The fix (prose math prompt ending in \boxed{}), no code prefill, Qwen3.x non-thinking) is now in the code and logged as D28. The 26 night items have not been regenerated on CPU; the only math generations with the fixed prompt are those of the descriptive TPU run (Section 7).
2. **Unregistered math answer extraction (D24).** The registration scores the boxed answer and treats unparseable math output as UNVERIFIED (fail-closed). The scorer (`math_check.extract_final_answer`) instead tries the last \boxed, then a #### line, then the last number anywhere in the text. With no boxed output, all 26 math items were scored by the last number in v3.3.0, and the passes it produced came from text ending in Python comments, a `print` call or a `return` statement. We froze the registered boxed-only rule before any math regeneration (`docs/FREEZE_ADDENDUM_2026-10-08_D25.md`), and in v3.3.1 it is the scoring default; the fallback is opt-in only (D28).
3. **Rust harness (fixed in v3.3.1, D27).** MultiPL-E has no reference solutions, so the Rust harness was first checked only by typechecking test bodies (D17), which did not catch that no Rust program could link inside the sandbox. v3.3.0 reported the resulting 0/26 as a result; it was an artefact. The harness was also open to a nonce forgery. Both are fixed; the fixed harness passes 25/25 hand-written correct solutions and fails 9/9 wrong ones on the night tasks (`results/gwenlaya_v4/rust_harness_validation.json`).
4. **Python harness environment (fixed, D29).** On the re-scoring machine a uv-managed interpreter could not start inside bubblewrap; fixed. The Python result is unchanged.
5. **Gate not recorded.** The `gate` field is null in every scored row, so no registered VERIFIED exists in this run and VERIFIED precision against hidden checks (H7) cannot be computed.
6. **Hidden tests in the HumanEval+ gate payload (D25).** The task builder restricted the HumanEval+ gate by rewriting only the loop header (`zip(inputs[:1], results[:1])`), so the complete hidden `inputs` and `results` lists stayed in every HumanEval+ gate payload (11/11 E-night rows, 161/161 primary rows; four primary items also carried a reference implementation). A candidate executed in the same process as `check()` could have read them, and any component that reads the gate payload would have seen the hidden answers. MBPP+, Rust and math gate payloads were correctly restricted. The statement that the gate “never sees the scoring checks” was therefore false for HumanEval+ in the night files. **No reported number changes:** the L2 run executed no gate. The builder now rewrites the literals themselves with an AST transform, fails closed, and is covered by a test asserting that no hidden element appears in a gate payload.

The task files were rebuilt with prompts and hidden checks unchanged (E-night 80 items; primary E 1,536 items after 5 fail-closed exclusions; hashes in Section 9).

What “passed” means in this paper. An item is PASS_HIDDEN only if it passed the named hidden checks: for Python, the full EvalPlus test suite run in a bubblewrap sandbox; for Rust, compilation with `rustc` and all `assert_eq! / assert_ne!` lines of the MultiPL-E test, run in a separate sandbox (D27); for math, a sympy/numeric equivalence check between the gold answer and the last `\boxed{}` answer, with no `\boxed` counted as UNDECIDED (D28). It means nothing more. It does not mean the solution is correct on other inputs.

6 TPU feasibility finding

Because the GPU slot was unavailable, we tested whether a Cloud TPU v5e could replace it. Two smoke runs (run #1) were made on a single-chip v5litepod-1 (logs under `gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/tpu_smoke/gwenlaya-tpu-smoke-215529/`). The first, on a spot VM, was preempted after about 4 minutes with no results. The second, on demand, gave the results below. The Qwen3.5 item is **updated in v3.3.1**: it described run #1 only, and run #3 (Section 7) shows that Qwen3.5 does generate on one v5e chip.

- **Qwen2.5 works.** Qwen/Qwen2.5-Coder-1.5B-Instruct loaded and generated with vLLM [9] 0.31.0’s TPU backend (load 208.9 s; 442 completion tokens in 0.61 s, 724.6 tok/s batched throughput; `res_qwen25.json`). The GWAYA gate on that VM returned UNVERIFIED for every item because bubblewrap was not available there, i.e. it failed closed as designed; no correctness number came out of this run.
- **Qwen3.5 generates on TPU with lazy compilation (updated).** In run #1, Qwen/Qwen3.5-4B resolved to Qwen3_5ForConditionalGeneration but did not finish loading/compiling within the 1,500 s step limit (exit 124). Qwen3.5 is not JAX-native in tpu-inference and runs through the PyTorch fallback; it runs when the up-front precompilation is skipped (`SKIP_JAX_PRECOMPILE=1`). With that setting, Qwen3.5-2B and Qwen3.5-4B generated all 80 night tasks in run #3 (Section 7); in run #2 Qwen3.5-0.8B also generated 16 night tasks (`results/gwenlaya_v4/tpu_run2/status.jsonl`); run #2 scores are not used, because its Python scoring was invalid (D29).
- **No TPU LoRA path was found for Qwen3.5 dense or Qwen3.8.** Serving Qwen2.5 with `enable_lora` failed at engine initialisation. The JAX training stacks we inspected list Qwen2/Qwen3 (Tunix 0.1.7) and Qwen2.5/Qwen3 plus Qwen3.5 mixture-of-experts scripts (MaxText), but no dense Qwen3.5 or Qwen3.8 path. GwenLaya’s training steps (Laya head, SFT) therefore stay on the GPU/CPU plan. This is unchanged in v3.3.1.

7 Descriptive TPU run (not a registered arm)

What ran. TPU run #3 used one on-demand TPU v5e chip (v5litepod-1, us-west4-a), vLLM-TPU 0.31.0, bf16 weights, greedy decoding (temperature 0), `max_tokens` 1024, and non-thinking mode, with the fixed prompts of D28. It generated all 80 night tasks (Python 28, Rust 26, math 26) for Qwen/Qwen3.5-2B, Qwen/Qwen3.5-4B and Qwen/Qwen2.5-Coder-1.5B-Instruct. Files: `results/gwenlaya_v4/tpu_run3/`.

Scoring. Scoring on the TPU host was *invalid for Python*: sandboxed checks launched from inside the vLLM process failed for every item (all 28 Python items FAILED for each model, `status.jsonl`). All TPU generations were therefore re-scored on our machine with the same checkers (`*.rescored.jsonl`, `summary.json`). For Rust and math the re-scored status equals

the TPU-host status on every item (26/26 and 26/26 for each model); for Python it does not. A math answer truncated at 1,024 tokens has no `\boxed` answer and counts as UNVERIFIED (`finish_reason = length` in the re-scored files). As in Section 5, a pass means the checks run by the scorer passed; no gate and no confidence score were recorded.

Table 4: Descriptive TPU run #3 (one v5e chip, bf16, greedy, one sample; not a registered arm, not a test). Counts of items passing the checks (status VERIFIED in the re-scored files) out of 28 Python, 26 Rust and 26 math items; “trunc.” = math items cut at 1,024 tokens (all UNVERIFIED). Load, generation time, completion tokens and tok/s are the run’s own SUMMARY fields (`status.jsonl`); tok/s is completion tokens over generation time for all 80 prompts submitted together, not single-stream speed.

Model	Python	Rust	Math	trunc.	load s	gen. s	tokens	tok/s
Qwen3.5-2B	11/28	6/26	16/26	9	106.2	99.7	25,568	256.3
Qwen3.5-4B	20/28	13/26	16/26	9	124.3	117.4	25,852	220.2
Qwen2.5-Coder-1.5B-Instruct	18/28	15/26	8/26	4	186.3	8.7	18,219	2,094.2

Reading. Qwen3.5-2B in bf16 on TPU has the same Python (11/28) and Rust (6/26) pass counts as the q4_K_M CPU night arm. The counts match, but not every item does: the outcome agrees on 22 of 28 Python and 24 of 26 Rust items (comparing `night_L2fix/rows.jsonl` with `res_qwen35_2b.rescored.jsonl`). We read this as a consistency check of the re-scored pipeline across hardware and quantisation, not as a test of anything. Math cannot be compared: the CPU math generations are invalid (D28), while the TPU math generations used the fixed prompt; for both Qwen3.5 models 9 of the 26 math items were truncated at the 1,024-token limit, so their math counts are lower bounds under this budget. Qwen2.5-Coder-1.5B is JAX-native in tpu-inference and its generation throughput was much higher than that of the Qwen3.5 models, which run through the PyTorch fallback (Table 4).

What this run is not. It has one sample per item, no log-probabilities, no gate, no calibration and no confidence score, so it says nothing about H1–H7. It was not a pre-registered arm and the model set was not registered. The benchmarks are public and the models have probably seen them in pretraining. Differences between the three models are single-sample counts on 26–28 items each and are not tested.

8 Actual GCP spend

Read from `/mnt/data/home/xavkal/gwaya-data/spend_ledger.jsonl` (5 lines):

Time	Item	Seconds	USD in ledger
2026-10-07 21:54	TPU smoke, attempt 1 (v5litepod-1 spot, us-west4-a; preempted)	240	0.0329 (catalog)
2026-10-07 22:38	TPU smoke (v5litepod-1 on demand, us-central1-a)	2,466	0.8220 (catalog)
2026-10-08 00:35	L1 upload to data lake (6 objects)	13	0.0
2026-10-08 07:09	TPU run #2 (v5litepod-1 on demand, us-central1-a)	1,073	0.3577 (catalog)
2026-10-08 07:36	TPU run #3 (v5litepod-1 on demand, us-west4-a; Section 7)	982	0.3273 (catalog)

The recorded USD total is \$1.54 (sum of `usd_estimate` = 1.5399), all from TPU use, priced from Cloud Billing Catalog list prices (spot v5e \$0.494237 per chip-hour; on-demand v5e \$1.20

per chip-hour, as recorded in each line’s `price_source`). The two smoke-test lines originally lacked `usd_estimate` and `resource` and were completed on 2026-10-08. Run #2 is in the ledger; its outputs are not used in this report (TBD). The night’s GPU (\$2.00) and endpoint (\$2.00) caps were not used. Local CPU work, including all re-scoring, cost nothing on GCP. These are list-price estimates; the billed amount is TBD until the billing export is read.

9 Reproducibility

Code. Repository branch `night/2026-10-07` (from `main`, released as v3.2.0 at 825d1ba); the first draft of this paper is commit 9b78e18; `numbers_v4.json` was regenerated for this revision (all numbers of the first draft unchanged, per-domain and boxed-only numbers added) and is committed with it. Revision v3.3.1 is on branch `fix/math-and-rescore`: the Rust harness fix is PR #4 (merged, 8cb37ad), the math and Python-sandbox fixes are commit 2c1f6b1, and `numbers_v4.json`, `tables_v4.tex` and the figures were regenerated from the re-scored rows `night/cache/L2fix/rows.jsonl` (a copy is `results/gwenlaya_v4/night_L2fix/rows.jsonl`; the `generations` file is unchanged). llama.cpp release b11476 (CPU, asset sha256 2cda5ff9...4b5e); Qwen3.5-2B-Q4_K_M.gguf from unsloth/Qwen3.5-2B-GGUF revision f6d5376b..., sha256 aaf42c8b...9223; 4B: revision e87f1764..., sha256 00fe7986...11a4 (all in `L0_env.json`).

Frozen inputs (sha256).

File	sha256
<code>experiments/night/manifest_eval.json</code>	bd1684dadcf38328fb6fab7619e5ea2f4fbc5ccc0e31022e059360b2d8c615e0
<code>tasks_E_night.jsonl</code> (80 items)	e842edd13951b4082c4495f7bcc845b032989f4a457e0c5d6e9c44f2b72cd679
<code>tasks_E_primary.jsonl</code> (1,541 items)	971f44c61bad5044f8487383f8a3ea27bc2c292c565a6ff8ef1254fce1cc7097
<code>docs/FREEZE_ADDENDUM_NIGHT_2026-10-07.md</code>	ac2970983ef45eb027de65194d42cadd41483dfd35cd2a866efc64247b02376c
<code>decontam_report.json</code>	d128e06c5949c86c29537f71a1d547ef953be6aa22f7bc814a280ec4d48f1907
<code>night/cache/L2score/rows.jsonl</code>	1e35fac792d29d8f82ce05f5cbad0d615bb4db2a6a707d157edff75da0fa69c6
<code>night/cache/L2score/gens.jsonl</code>	03f6e2b6af9cebd65d8488b9a35a1f7d3bc70b566397379edc3d95f5291963db
<code>papers/numbers_v4.json</code> (v3.3.0)	e5488f6c66419aa99971eef25c10f1f076b0595d1b9c02db84e007591d32b07c
<code>night / cache / L2fix / rows . jsonl</code> (v3.3.1 re-score)	a5e72614681a81acf379050270677eead4844e4833494c32f7695c485cae70a8
<code>night / cache / L2fix / gens . jsonl</code> (= L2score)	03f6e2b6af9cebd65d8488b9a35a1f7d3bc70b566397379edc3d95f5291963db
<code>papers/numbers_v4.json</code> (v3.3.1)	952bfc771498bf0914a816935671b957f3ae89a189a3c00c99ddbf26a5c6eb74
<code>results/gwenlaya_v4/rust_harness_validation.json</code>	93291a5a56c050d15fbb98a501e16d575251602e30507e563ff610f24583e8d8
<code>results / gwenlaya _ v4 / tpu _ run3 / summary.json</code>	e38310f6b4e7869c0f84a3cfad874e0820cd77a29f81e173f6e4c0cc80f94648
<code>tasks_E_night.d25.jsonl</code> (80, D25)	504909fc74dc2af8fb3f2491a475df07de8e95e6fdf0f4f69bc22352e95e0952
<code>tasks_E_primary.d25.jsonl</code> (1,536, D25)	2143110c78a3a9f57599dce77021cf4571fc57cf2584017f16ebdd08d5acba66

HF dataset revisions are in the freeze addendum (MATH-500 6e4ed1a2, humanevalplus d32357cf, mbppplus b2d74c91, MultiPL-E 28441b60). The `rows.jsonl` and `gens.jsonl` hash chains verify (`numbers_v4.json` meta). The L2 run used the original task files; the `.d25` files (gate pay-

loads rebuilt, prompts and hidden checks unchanged) are for all later runs and are described in docs/FREEZE_ADDENDUM_2026-10-08_D25.md.

Data lake. `gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/night/2026-10-07/` holds the manifest, both task files, the sizing file, the addendum and the decontamination report. The TPU logs are under `.../gwenlaya_v4/tpu_smoke/`. The scored L2 rows and generations are *not yet* in the lake (they are on the local data disk under `night/cache/L2score/` and `night/cache/L2fix/`; the re-scored rows and the TPU run #3 files are also in the repository under `results/gwenlaya_v4/`); the registration requires results tables to come from `gwenlaya_v4/results/`, so uploading them is an open item and this is recorded as a limitation.

Commands. With the environment of `night/run_L2.sh` (`GWAYA_DATA_ROOT`, `HF_HOME`, `PYTHONPATH`):

```
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 \
  --backend openai --backend-url http://127.0.0.1:8091/v1 \
  --tasks $NIGHT/results/tasks_E_night.jsonl --models qwen3.5-2b-q4_K_M \
  --quants q4_K_M --arms base --mode generate --no-vram --cpu-pid <llama-server pid> \
  --out $NIGHT/cache/L2
python scripts/run_study.py ... --mode score --out $NIGHT/cache/L2score # v3.3.0
cp $NIGHT/cache/L2/gens.jsonl $NIGHT/cache/L2fix/ # v3.3.1: re-score only, no new generation
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 \
  --backend openai --backend-url http://127.0.0.1:8091/v1 \
  --tasks $NIGHT/results/tasks_E_night.jsonl --models qwen3.5-2b-q4_K_M \
  --quants q4_K_M --arms base --mode score --no-vram --out $NIGHT/cache/L2fix
python scripts/analyze_study.py --results $NIGHT/cache/L2fix/rows.jsonl $NIGHT/cache/L2fix/gens.jsonl \
  --tasks $NIGHT/results/tasks_E_night.jsonl --ledger $GWAYA_DATA_ROOT/spend_ledger.jsonl \
  --n-boot 10000 --seed 0
# writes numbers_v4.json, tables_v4.tex, figures_v4/; --results must name one scored
# directory only, because the gens.jsonl copies are byte-identical and would be counted twice
python scripts/data/regen_gate_payloads_d25.py --results $NIGHT/results # D25 task files
```

The llama-server ran on 127.0.0.1:8091 with `-t 4 -parallel 2` (D14).

10 Limitations

- No pre-registered test ran. Nothing here supports or refutes H1–H12, H14 or H15.
- The only scored arm is unregistered, single-seed, $n = 80$, on CPU, with no gate recorded.
- The night math generations are invalid (prompt-path defect, D28); under the registered boxed-only rule none passes. Python ($n = 28$) and Rust ($n = 26$) are usable descriptive results only; the Python AUROC interval includes 0.5, and Rust has 6 passing items.
- The v3.3.0 Rust result was a harness artefact (D27). The fixed Rust harness was validated after the generations existed, on 25 hand-written correct and 9 hand-written wrong solutions written for this purpose; it has not been checked against an independent reference set, and one of the 26 night Rust tasks has no reference solution in the validation.
- The TPU run (Section 7) is unregistered, single-sample, without log-probabilities, gate or calibration; its TPU-host Python scores were invalid and all its rows were re-scored off the TPU host. Math truncations at 1,024 tokens count as UNVERIFIED.
- **Contamination.** All absolute scores are on public sets (HumanEval+, MBPP+, MultiPL-E, MATH-500) that Qwen3.5 has probably seen in pretraining; they are not clean capability estimates, and only paired within-model deltas are claimed by the registration. Decontamination (D18) covered only training/calibration data against E and cannot detect pretraining

exposure. The contamination-stratified analysis asked for by review finding F12 (for example MBPP+ versus HumanEval+ items, or a post-cutoff anchor such as LiveCodeBench, which D10 dropped) was not done: TBD.

- The HumanEval+ gate payloads of the night task files contained the hidden tests (D25). No gate ran in L2, so no result is affected, but any earlier claim that the gate never saw the scoring checks was false for those files.
- Several deviations (D22–D30) were written after the L2 generations existed; none was written after a registered test, because none ran.
- Even with all arms present, $n = 80$ gives H1 power of about 0.05 at its MEI; the night design could at best produce an inconclusive H1.
- Cost is server CPU-seconds on a machine shared with foreign jobs and includes idle threads; it is not comparable to the registered L4 GPU-seconds.
- The USD spend is a list-price estimate, not the billed amount, and H13 holds largely by construction of the ledger.
- Scored rows are not yet mirrored to the data lake as the registration requires.
- Lean 4, the training arms, quantization and the endpoint were not run. The TPU findings are from one chip and one vLLM version.

11 Next steps

Regenerate the 26 night math items on the CPU arm with the fixed prompt (D28) and score them boxed-only; read the billed TPU amounts from the billing export; generate the 4B arm and the GL/B3/B5 arms on the same 80 items with the .d25 task files and the fixed harnesses; build the cross-fitted scores file; upload results to the data lake; and rerun `analyze_study.py`. Only then can H1, H3, H2, H7 and H4–H6 move off “not run”. A null or negative result will be reported in the same form as this one.

References

- [1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [2] Xavier Callens. GWAYA v3: A fail-closed verification gate for LLM-generated code. Zenodo, <https://doi.org/10.5281/zenodo.23121788>, 2026. Version 3.1.0; its +8.0 point claim is retracted.
- [3] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 2023.
- [4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- [5] ggml-org. llama.cpp. <https://github.com/ggml-org/llama.cpp>, 2026. Release b11476, CPU build.

- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.
- [7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [8] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.
- [12] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

A Machine-generated tables

The tables on the next page are `papers/tables_v4.tex` as written by `scripts/analyze_study.py` from `papers/numbers_v4.json` (content unedited, font reduced). “CPU-s total” is server CPU-seconds, which include idle-spinning worker threads on a machine shared with three foreign CPU-bound jobs (D14, D21). In these tables “acc.” and “pass rate” are the `PASS_HIDDEN` rate (`rows.jsonl score = VERIFIED`, i.e. passed the hidden checks), not the registered gate `VERIFIED`. CWR counts every `UNDECIDED` answer as wrong, so it is an upper bound. The arm table holds the pooled 80 items and is a sensitivity analysis only (Section 5). The hypothesis-status table is the script’s machine status; Table 2 is the reviewed verdict. Since v3.3.1 math is scored boxed-only by default (D28), so the two math rows and the two pooled rows are equal.

Table 5: Cached generations on the E-night set (greedy, one seed). Tokens are mean completion tokens.

model	domain	n	mean tok.	CPU-s total	finish reasons
qwen3.5-2b-q4_K_M	math	26	371.0	9911	length:4, stop:22
qwen3.5-2b-q4_K_M	python	28	118.8	3625	stop:27, length:1
qwen3.5-2b-q4_K_M	rust	26	98.5	2963	stop:26

Table 6: Single unregistered arm **base**, per domain (descriptive, not a test). Pass = passed the hidden checks (PASS_HIDDEN), not the registered gate VERIFIED. Confidence = raw exp(mean log-prob). 95% cluster bootstrap CI. Cost unit: cpu_seconds. TBD = undefined (e.g. no passing item).

domain		status	n	pass rate	mean conf.	ECE	AUROC	cost / pass
math		INVALID prompt path (D24/D28); boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
python		headline	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
rust		validated harness (D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
math (boxed-only)		INVALID prompt path; registered boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
pooled (boxed-only math)		sensitivity	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]
pooled		sensitivity (mixed)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

Table 7: Arms on the paired E-night items (percent; 95% cluster bootstrap CI). Cost unit: cpu_seconds. TBD = not run or not available.

arm	acc.	coverage	CWR	ans. acc.	cost / correct
base	21.2 [12.5, 30.0]	100.0 [100.0, 100.0]	78.8 [70.0, 87.5]	21.2 [12.5, 30.0]	970.5 [629.2, 1647.9]

Table 8: Hypothesis status for the night run. Verdicts follow the pre-registered rules; primary Holm with $m = 2$.

H	role	result	detail
H1	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H2	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H3	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H4	secondary	TBD	no cross-fitted scores file (no scores file)
H5	secondary	TBD	no cross-fitted scores file (no scores file)
H6	secondary	TBD	no cross-fitted scores file (no scores file)
H7	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H8	exploratory_only_if_G1	TBD	needs G1 (no GPU slot)
H9	deferred	deferred	deferred (D8)
H10	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H11	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H12	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H13	run	not refuted	
H14	only_if_E1	TBD	needs E1 (D12)
H15	offline_test	TBD	offline kill/resume test, see tests/test_run_study.py